CS 4433: Project 3 Report

Skyler Dooley & Alessandra (Ally) Giovenco

Guide / Execution Instructions

To run this project, Python and PySpark must be installed. The required libraries are pyspark, csv, and the standard Python modules used in the scripts. PySpark can be installed with `pip install pyspark`. The code also requires a Java runtime (JDK 8 or later) and a Hadoop directory for Windows users; the scripts set the necessary environment variables (PYSPARK_PYTHON, PYSPARK_DRIVER_PYTHON, and HADOOP_HOME) so Spark can start correctly. Once PySpark is installed and Java is available, all scripts can be executed directly with Python.

Before running any queries, the datasets must be generated. For Problem 1, the dataset generator is located in problem1/generateDatasets.py. Running this script with `python generateDatasets.py` creates the three required files: MetaEvent.csv, MetaEventNoDisclosure.csv, and ReportedIllnesses.csv. These files are written into the datasets/ directory and are used by all five Problem 1 queries. After the datasets are created, each query script (query1.py through query5.py) can be run individually using `python queryX.py` inside the Problem 1 directory. Each script initializes PySpark, loads the appropriate CSV files, performs the required RDD transformations, and prints the results to the console. The data generation file can be found below:

```python
# problem1/generateDatasets

import csv
import os
import random

# Parameters
NUM_PEOPLE    = 500_000
NUM_TABLES    = 10_000
SICK_FRACTION = 0.05
OUTPUT_DIR    = "datasets"
SEED          = 42

os.makedirs(OUTPUT_DIR, exist_ok=True)
random.seed(SEED)

meta_event_path       = os.path.join(OUTPUT_DIR, "MetaEvent.csv")
no_disclosure_path    = os.path.join(OUTPUT_DIR, "MetaEventNoDisclosure.csv")
reported_path         = os.path.join(OUTPUT_DIR, "ReportedIllnesses.csv")

print("Generating datasets …")

with open(meta_event_path,    "w", newline="") as f_meta, \
```

```
        open(no_disclosure_path, "w", newline="") as f_nod, \
        open(reported_path,      "w", newline="") as f_rep:

    w_meta = csv.writer(f_meta)
    w_nod  = csv.writer(f_nod)
    w_rep  = csv.writer(f_rep)

    # Headers
    w_meta.writerow(["pi_id", "pi_name", "pi_table", "pi_test"])
    w_nod .writerow(["pi_id", "pi_name", "pi_table"])
    w_rep .writerow(["pi_id", "pi_test"])

    sick_count = 0

    for i in range(1, NUM_PEOPLE + 1):
        pid     = i
        name    = f"Person_{pid:07d}"
        table   = f"Table_{random.randint(1, NUM_TABLES):04d}"
        test    = "sick" if random.random() < SICK_FRACTION else "not-sick"

        w_meta.writerow([pid, name, table, test])
        w_nod .writerow([pid, name, table])

        if test == "sick":
            w_rep.writerow([pid, test])
            sick_count += 1

        if i % 100_000 == 0:
            print(f"  {i:,} / {NUM_PEOPLE:,} rows written …")

print("\n Done")
print(f"Meta-Event             : {meta_event_path}  ({NUM_PEOPLE:,} rows)")
print(f"Meta-Event-No-Disc.    : {no_disclosure_path}  ({NUM_PEOPLE:,} rows)")
print(f"Reported-Illnesses     : {reported_path}  ({sick_count:,} rows, "
      f"{sick_count/NUM_PEOPLE*100:.1f}% sick)")
```

For Problem 2, the dataset generator is located in problem2/generateDatasets.py. Running `python generateDatasets.py` produces the Customers.csv and Purchases.csv files. These files are also written into the shared datasets/ directory. After the datasets are created, each query script (query1.py through query5.py) can be run individually using `python queryX.py` inside the Problem 2 directory. Query 1 automatically writes the filtered output to T1.csv, which is required by Queries 2–4. Query 3 writes its grouped

results to T3.csv. All other queries print their results directly to the console. The data generation file can be found below:

```python
# problem2/generateDatasets

import csv
import os
import random
import string

# Parameters
NUM_CUSTOMERS  = 50_000
NUM_PURCHASES  = 5_000_000
OUTPUT_DIR     = "datasets"
SEED           = 42

os.makedirs(OUTPUT_DIR, exist_ok=True)
random.seed(SEED)

customers_path = os.path.join(OUTPUT_DIR, "Customers.csv")
purchases_path = os.path.join(OUTPUT_DIR, "Purchases.csv")

print("Generating Customers and Purchases datasets …")

# Generate Customers.csv
with open(customers_path, "w", newline="") as f_cust:
    w = csv.writer(f_cust)
    w.writerow(["CustID", "Name", "Age", "Address", "Salary"])

    for cid in range(1, NUM_CUSTOMERS + 1):
        name = ''.join(random.choices(string.ascii_letters,
k=random.randint(10, 20)))
        age = random.randint(18, 100)
        address = ''.join(random.choices(string.ascii_letters + " ", k=30))
        salary = round(random.uniform(1000, 10000), 2)

        w.writerow([cid, name, age, address, salary])

        if cid % 10_000 == 0:
            print(f"  Customers: {cid:,} / {NUM_CUSTOMERS:,}")

# Generate Purchases.csv
with open(purchases_path, "w", newline="") as f_pur:
    w = csv.writer(f_pur)
```

```
    w.writerow(["TransID", "CustID", "TransTotal", "TransNumItems",
"TransDesc"])

    for tid in range(1, NUM_PURCHASES + 1):
        cust_id = random.randint(1, NUM_CUSTOMERS)
        total = round(random.uniform(10, 2000), 2)
        num_items = random.randint(1, 15)
        desc = ''.join(random.choices(string.ascii_letters + " ",
k=random.randint(20, 50)))

        w.writerow([tid, cust_id, total, num_items, desc])

        if tid % 500_000 == 0:
            print(f"  Purchases: {tid:,} / {NUM_PURCHASES:,}")

print("\n Done")
print(f"Customers  : {customers_path}  ({NUM_CUSTOMERS:,} rows)")
print(f"Purchases  : {purchases_path}  ({NUM_PURCHASES:,} rows)")
```

For the optional Bonus Problem 3, the MLlib script can be run after the Customers and Purchases datasets have been generated. The code is contained in a standalone file (problem3.py) inside the Problem 3 directory. To execute it, run `python problem3.py` from the command line inside the Problem 3 directory. The script initializes Spark, loads the datasets, assembles the machine-learning features, performs an 80/20 train-test split, trains a regression model to predict purchase totals, and prints the evaluation metrics (RMSE and MAE) directly to the console. No additional setup is required beyond installing PySpark and generating the dataset for problem 2.

```shell
cd problem3
python problem3.py
```

Documentation of the Project (including screenshots)

Query 1.1 Ally (and datagen) -

Query 1 loads the Meta-Event dataset and filters the RDD to return only the people whose test result is marked as “sick.” The script parses each row into fields and applies a simple filter on the pi_test attribute before printing the matching records.

```
First 10:
  id=       6  name=Person_0000006        table=Table_0055  test=sick
  id=      78  name=Person_0000078        table=Table_0063  test=sick
  id=      96  name=Person_0000096        table=Table_0044  test=sick
  id=      98  name=Person_0000098        table=Table_0029  test=sick
  id=     144  name=Person_0000144        table=Table_0077  test=sick
  id=     153  name=Person_0000153        table=Table_0010  test=sick
  id=     175  name=Person_0000175        table=Table_0006  test=sick
  id=     182  name=Person_0000182        table=Table_0070  test=sick
  id=     212  name=Person_0000212        table=Table_0034  test=sick
  id=     288  name=Person_0000288        table=Table_0062  test=sick
```

Query 1.2 Skyler -

Query 2 reads the No-Disclosure file and the Reported-Illnesses file, parses both into RDDs keyed by person ID, and performs an RDD join. This join returns all people who appear in the No-Disclosure dataset and are also listed as sick in the Reported-Illnesses file, effectively reconstructing the set of known sick attendees.

```
Total reported sick found in No-Disclosure: 226

First 10:
  id=144  name=Person_0000144  table=Table_0077  test=sick
  id=153  name=Person_0000153  table=Table_0010  test=sick
  id=442  name=Person_0000442  table=Table_0074  test=sick
  id=723  name=Person_0000723  table=Table_0095  test=sick
  id=912  name=Person_0000912  table=Table_0058  test=sick
  id=982  name=Person_0000982  table=Table_0073  test=sick
  id=1364  name=Person_0001364  table=Table_0014  test=sick
  id=1418  name=Person_0001418  table=Table_0079  test=sick
  id=1440  name=Person_0001440  table=Table_0039  test=sick
  id=1655  name=Person_0001655  table=Table_0001  test=sick
```

Query 1.3 Ally -

Query 3 identifies healthy people who shared a table with at least one sick person. The script first extracts all tables containing sick individuals, broadcasts that set, and then filters the full Meta-Event dataset to return only healthy people whose table appears in the sick-table set. Duplicate entries are removed to ensure each healthy person is returned once.

```
Healthy people at risk (sitting with sick person): 4,323

First 10:
  id=       2  name=Person_0000002        table=Table_0095  test=not-sick
  id=       3  name=Person_0000003        table=Table_0029  test=not-sick
  id=       4  name=Person_0000004        table=Table_0014  test=not-sick
  id=       5  name=Person_0000005        table=Table_0070  test=not-sick
  id=       8  name=Person_0000008        table=Table_0065  test=not-sick
  id=      12  name=Person_0000012        table=Table_0076  test=not-sick
  id=      13  name=Person_0000013        table=Table_0001  test=not-sick
  id=      15  name=Person_0000015        table=Table_0044  test=not-sick
  id=      18  name=Person_0000018        table=Table_0049  test=not-sick
  id=      20  name=Person_0000020        table=Table_0006  test=not-sick
```

Query 1.4 Skyler-

Query 4 groups the Meta-Event dataset by table and computes two values for each table: the number of people seated there and a flag indicating whether the table is fully healthy or of concern. The flag is set to “healthy” only if all people at the table are not-sick; otherwise it is marked as “concern.”

```
Total tables: 100

First 10:
  table=Table_0082  count=57  flag=concern
  table=Table_0095  count=50  flag=concern
  table=Table_0014  count=59  flag=concern
  table=Table_0070  count=45  flag=concern
  table=Table_0072  count=46  flag=concern
  table=Table_0076  count=54  flag=concern
  table=Table_0044  count=56  flag=concern
  table=Table_0045  count=42  flag=concern
  table=Table_0069  count=53  flag=concern
  table=Table_0080  count=48  flag=concern
```

Query 1.5 Ally -

Query 5 performs the cross-file version of the close-contact analysis. Using the No-Disclosure dataset and the Reported-Illnesses file, the script identifies which tables contain sick individuals and then returns all people not listed as sick who sit at those tables. This produces the set of healthy attendees who were exposed, without relying on health information inside the main event file.

```
Healthy people at risk (close contact with sick person): 4,323

First 10:
  id=       2  name=Person_0000002        table=Table_0095
  id=       3  name=Person_0000003        table=Table_0029
  id=       4  name=Person_0000004        table=Table_0014
  id=       5  name=Person_0000005        table=Table_0070
  id=       8  name=Person_0000008        table=Table_0065
  id=      12  name=Person_0000012        table=Table_0076
  id=      13  name=Person_0000013        table=Table_0001
  id=      15  name=Person_0000015        table=Table_0044
  id=      18  name=Person_0000018        table=Table_0049
  id=      20  name=Person_0000020        table=Table_0006
```

Query 2.1 Skyler (and datagen)-

Query 1 loads the Purchases dataset and filters out all transactions where the purchase total exceeds $100. The remaining rows are written to T1.csv, which serves as the input for the subsequent queries.

query1.py M T1.csv U

problem2 > problem2 > T1.csv > T1.csv > data

```
TransID,CustID,TransTotal,TransNumItems,TransDesc
44,656,96.27,14,zhtoBrBKRPyjNRhlrbqkuCZLNUWJ
55,3137,52.58,15,bVjXhbqTyXvrxhVEPHbWnVCHSBeevjKy
76,3779,78.1,15,urakxEFTiVPDefnqDNpveubmdsEepAVKPaZewVHLltSJlg
124,1374,45.27,11,SxGjilLK EhukNbDxvyDklhE
137,409,89.21,5,HKOKKFcfizMZEQXVqzYTnGOlsvtT kAIhkPUCnz
153,3194,45.59,6,UehwktIfVVXFDeosBqGTaDZw
155,2449,28.66,6,NqeIAbhnzhmPxZNirRXMMEEjAntBPqkGY
163,4959,98.93,13,WnacGFbpQHwXqLOSDoNtVyW UTQsFvhjRpbhIpEdWSgis
185,3749,98.98,9,dvxGgLmFIaJIhVXXslJywpkHhZvQxrezjVfTTFxcukRLVR
216,1349,81.95,2,CAJGTrEIcHNRV HHQChfjEbD
263,2810,70.61,8,BcXedpjHMx AgsfKFmHcxLnPXHmlSquSNIHUOI
265,2158,47.1,1,hS iKvdrqRGVzlEqzgoAlLNohEYRfbRPlm
270,434,72.87,11,gf oIUanMmduKKZMwfWXmsbtxtRpkvwQmGf
304,3639,47.64,6,QxyXaislCTbcJFsKXWJwWHUnM
```

```
Query 1 complete.
Rows in T1: 2,186
```

Query 2.2 Ally-

Query 2 reads T1.csv and groups the filtered purchases by the number of items in each transaction. For each group, the script computes the median, minimum, and maximum purchase totals using SparkSQL functions. The results are displayed directly to the client.

```
T1 row count: 2,186
root
 |-- TransID: integer (nullable = true)
 |-- CustID: integer (nullable = true)
 |-- TransTotal: double (nullable = true)
 |-- TransNumItems: integer (nullable = true)
 |-- TransDesc: string (nullable = true)


Query 2 Results - Grouped by Number of Items Purchased:
================================================================
+-------------+-------------+----------+----------+-------------+
|TransNumItems|median_amount|min_amount|max_amount|num_purchases|
+-------------+-------------+----------+----------+-------------+
|1            |59.61        |11.06     |99.79     |144          |
|2            |57.01        |10.42     |99.74     |156          |
|3            |55.39        |10.24     |99.53     |130          |
|4            |56.79        |10.37     |99.05     |138          |
|5            |51.01        |10.11     |98.61     |160          |
|6            |54.45        |10.62     |99.36     |169          |
|7            |52.35        |10.64     |98.05     |153          |
|8            |61.37        |10.72     |99.1      |139          |
|9            |55.79        |10.41     |99.02     |164          |
|10           |54.17        |11.91     |99.68     |137          |
|11           |55.81        |12.37     |99.92     |153          |
|12           |54.15        |11.2      |99.54     |160          |
|13           |51.16        |12.23     |99.34     |128          |
|14           |51.29        |10.62     |99.43     |137          |
|15           |56.24        |10.38     |98.11     |118          |
+-------------+-------------+----------+----------+-------------+
```

Items	Median $	Min $	Max $	Count
1	59.61	11.06	99.79	144
2	57.01	10.42	99.74	156
3	55.39	10.24	99.53	130
4	56.79	10.37	99.05	138
5	51.01	10.11	98.61	160
6	54.45	10.62	99.36	169
7	52.35	10.64	98.05	153
8	61.37	10.72	99.10	139
9	55.79	10.41	99.02	164
10	54.17	11.91	99.68	137
11	55.81	12.37	99.92	153
12	54.15	11.20	99.54	160
13	51.16	12.23	99.34	128
14	51.29	10.62	99.43	137
15	56.24	10.38	98.11	118

Query 2.3 Skyler-

Query 3 joins T1 with the Customers dataset, filters for GenZ customers aged 18–21, and aggregates their total number of items purchased and total spending. The grouped results are written to T3.csv for later inspection.

T3.csv U

problem2 > problem2 > T3.csv > T3.csv > data

```
CustID,Age,total_items,total_spent
696,19,4,21.19
1502,20,5,49.65
3697,21,12,157.74
930,21,14,52.84
143,20,12,57.7
2945,19,3,69.99
3225,21,7,179.20999999999998
518,19,15,88.24
1511,19,15,95.44
1668,19,8,72.73
370,18,22,31.02
511,20,1,37.46
649,21,11,83.56
4346,20,4,37.22
```

```
Query 3 complete.
Rows in T3: 88
```

Query 2.4 Ally-

For query 4, it returned 0 customers who cannot cover their expenses. This is expected because T1 only contains purchases with TransTotal ≤ $100, and the maximum total expenses per customer in T1 is $296.62, which is well below the minimum salary of $1,000.11.

```
PS C:\Users\agiov\proj3-CS4433\problem2> python query4.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Customers loaded: 5,000 rows
T1 loaded: 2,186 rows

Customers who cannot cover their expenses: 0
==========================================================================
+------+------+-------+--------------+
|CustID|Salary|Address|total_expenses|
+------+------+-------+--------------+
+------+------+-------+--------------+


CustID         Salary   Total Expenses    Shortfall  Address
--------------------------------------------------------------------------
```

Bonus Task Skyler -

For the bonus task, we used the Customers and Purchases datasets from Problem 2 to train a regression model that predicts the purchase total (TransTotal). We selected the relevant numeric attributes—Age, Salary, and TransNumItems—and assembled them into a single MLlib feature vector. The full dataset was then split into an 80% training set and a 20% testing set. We trained a linear regression model on the training portion and applied it to the test set to generate predictions. To evaluate performance, we computed standard regression metrics including RMSE and MAE. These metrics indicate how closely the model's predictions match the actual purchase totals and provide a basis for assessing whether the chosen features and model were effective for this prediction task.

```
Model trained.
Coefficients: [-0.1015311562472043,0.0006375824854450445,-0.9339194557004995]
Intercept:    1013.6762271713101
+-------------------+-------+------------------+
|features           |label  |prediction        |
+-------------------+-------+------------------+
|[18.0,1046.23,9.0] |634.68 |1004.1104491813031|
|[18.0,1046.62,3.0] |206.39 |1009.7142145726754|
|[18.0,1046.62,6.0] |199.63 |1006.9124562055739|
|[18.0,1046.62,14.0]|1565.22|999.4411005599699 |
|[18.0,1193.12,3.0] |1579.57|1009.807620406793 |
|[18.0,1193.12,14.0]|373.77 |999.5345063940875 |
|[18.0,1279.79,8.0] |606.17 |1005.1932824023041|
|[18.0,1389.95,1.0] |1102.05|1011.8009546788043|
|[18.0,1389.95,14.0]|921.47 |999.6600017546978 |
|[18.0,1389.95,14.0]|1012.02|999.6600017546978 |
+-------------------+-------+------------------+
only showing top 10 rows


Evaluation Metrics:
  RMSE: 571.2484
  MAE : 493.8975
```

```
+------+------------------+
|CustID|             total|
+------+------------------+
|  3889|            296.62|
|  2878|            271.14|
|  3603|             261.6|
|  1376|            256.91|
|  1691|245.20999999999998|
+------+------------------+
only showing top 5 rows

+-----------+-----------+
|min(Salary)|max(Salary)|
+-----------+-----------+
|    1000.11|    9999.49|
+-----------+-----------+
```

After training and evaluating the regression model, the testing results produced an RMSE of 571.2484 and a MAE of 493.8975. These metrics indicate that the model's predictions deviate substantially from the true purchase totals. The MAE shows that, on average, predictions are off by roughly $494 per transaction, while the RMSE—being even larger—suggests that the model occasionally makes very large errors that heavily influence the overall score. Because purchase totals in the dataset range from $10 to $2000, errors of this magnitude represent a significant portion of the entire value range. This level of inaccuracy suggests that the model is not very effective at predicting purchase amounts. A likely reason for the poor performance is that the synthetic dataset is highly random, meaning the selected features (Age, Salary, and TransNumItems) do not strongly determine the purchase total, leaving the model with little meaningful structure to learn.

Resource Usage Statement

For this project, we relied exclusively on the Spark environment provided in the course Docker setup. All dataset generation, file loading, and execution of the Spark RDD, SparkSQL, and optional MLlib tasks were carried out directly within this local Spark installation, as required by the assignment. All PySpark code used for RDD transformations, SQL-based analytics, and the Bonus Problem 3 machine-learning workflow was written and executed by our team. We validated the correctness of each task by inspecting intermediate outputs, checking schema and row counts, reviewing model evaluation metrics, and confirming that the results aligned with the expected behavior described in the project specification. We did not use any external datasets, pre-built analytics code, or cloud-based Spark services; all computation was performed locally using the tools and infrastructure configured for this course.

Contribution Statement

We divided the project so that each teammate completed alternating tasks, ensuring an even balance between the simpler components and the more complex ones. This approach

allowed both team members to work on RDD-based tasks, SparkSQL tasks, and dataset generation in a fair and balanced way. The name of the teammate responsible for each task is listed in the corresponding file header. We reviewed each other's work, validated outputs together, and coordinated on debugging and final report preparation to ensure consistency across the entire project.